PRACTICAL NO.1

Title:

Write a Python program to manage the borrowing records of books in a library. Implement thefollowingfunctionalities:

•Compute the average number of books borrowed by all library members.

•Find the book with the highest and lowest number of borrowings in the library.

•Count the number of members who have not borrowed any books (denoted by a borrow count of 0).

•Display the most frequently borrowed book (i.e., the mode of borrow counts). After performing, determine the time and Space complexity of each operation.

Program Code:

```
from collections import Counter

 borrow_counts = [3, 0, 2, 5, 0, 3, 1, 4, 3, 2]

1. Compute average number of books borrowed

total = sum(borrow_counts)

average = total / len(borrow_counts)

print("Borrowing Records:", borrow_counts)

print("Average number of books borrowed:", average)

2. Find highest and lowest number of borrowings

highest = max(borrow_counts)

lowest = min(borrow_counts)

print("Highest number of borrowings:", highest)

print("Lowest number of borrowings:", lowest)
```

3. Count members who have not borrowed any books

```
zero_count = borrow_counts.count(0)
print("Number of members who borrowed 0 books:", zero_count)
```

4. Find the most frequently borrowed count (Mode)

```
frequency = Counter(borrow_counts)
mode = frequency.most_common(1)[0][0]
print("Most frequently borrowed count (Mode):", mode)
```

Output:

```
Borrowing Records: [3, 0, 2, 5, 0, 3, 1, 4, 3, 2]
Average number of books borrowed: 2.3
Highest number of borrowings: 5
Lowest number of borrowings: 0
Number of members who borrowed 0 books: 2
Most frequently borrowed count (Mode): 3
```

Practical No. 02

Title: In a company, employee salaries are stored in a list as floating-point numbers. Write a Python program that sorts the employee salaries in ascending order using the following two algorithms:

1. **Selection Sort:** Sort the salaries using the Selection Sort algorithm.
2. **Bubble Sort:** Sort the salaries using the Bubble Sort algorithm.

After sorting the salaries, the program should display the top five highest salaries in the company.

Program Code:

```
# Function for Selection Sort

def selection_sort(arr):

    n = len(arr)

    sorted_arr = arr.copy()

    for i in range(n):

        min_index = i

        for j in range(i + 1, n):

            if sorted_arr[j] < sorted_arr[min_index]:

                min_index = j

        sorted_arr[i], sorted_arr[min_index] = sorted_arr[min_index], sorted_arr[i]

    return sorted_arr

# Function for Bubble Sort

def bubble_sort(arr):

    n = len(arr)

    sorted_arr = arr.copy()

    for i in range(n):

        for j in range(0, n - i - 1):

            if sorted_arr[j] > sorted_arr[j + 1]:
```

```
                sorted_arr[j], sorted_arr[j + 1] = sorted_arr[j + 1], sorted_arr[j]
    return sorted_arr
# Main Program
salaries = []
n = int(input("Enter the number of employees: "))
for i in range(n):
    salary = float(input(f"Enter salary of employee {i + 1}: "))
    salaries.append(salary)
# Selection Sort
selection_sorted = selection_sort(salaries)
print("\nSalaries after Selection Sort (Ascending):")
print(selection_sorted)
# Bubble Sort
bubble_sorted = bubble_sort(salaries)
print("\nSalaries after Bubble Sort (Ascending):")
print(bubble_sorted)
# Display Top Five Highest Salaries
print("\nTop Five Highest Salaries:")
top_five = bubble_sorted[-5:] if len(bubble_sorted) >= 5 else bubble_sorted
top_five.reverse()
for i, salary in enumerate(top_five, start=1):
    print(f"{i}. {salary}")
```

Output:

Enter the number of employees: 7

Enter salary of employee 1: 35000

Enter salary of employee 2: 42000

Enter salary of employee 3: 28000

Enter salary of employee 4: 50000

Enter salary of employee 5: 39000

Enter salary of employee 6: 47000

Enter salary of employee 7: 31000

Salaries after Selection Sort (Ascending):

[28000.0, 31000.0, 35000.0, 39000.0, 42000.0, 47000.0, 50000.0]

Salaries after Bubble Sort (Ascending):

[28000.0, 31000.0, 35000.0, 39000.0, 42000.0, 47000.0, 50000.0]

Top Five Highest Salaries:

1. 50000.0

2. 47000.0

3. 42000.0

4. 39000.0

5. 35000.0

Practical No. -03

Title: Implementing a real-time undo/redo system for a text editing application using a Stack data Structure.The system should support the following operations:

•Make a Change: A new change to the document is made.

•Undo Action: Revert the most recent change and store it for potential redo.

•Redo Action: Reapply the most recently undone action.

•Display Document State: Show the current state of the document after undoing or

Redoing an action.

Program:

```
#include <iostream>

#include <stack>

#include <string>

using namespace std;


class TextEditor {

private:

    string document;

    stack<string> undoStack;

    stack<string> redoStack;


public:

    // Make a new change

    void makeChange(const string& newText) {

        // Save current state before making the change

        undoStack.push(document);
```

```
    document = newText;

    // Clear redo stack because a new change is made
    while (!redoStack.empty())
        redoStack.pop();

    cout << "Change made." << endl;
}

// Undo the most recent change
void undo() {
    if (undoStack.empty()) {
        cout << "Nothing to undo." << endl;
        return;
    }

    // Save current state for redo
    redoStack.push(document);

    // Restore previous state
    document = undoStack.top();
    undoStack.pop();

    cout << "Undo performed." << endl;
```

```
    }

    // Redo the most recently undone change
    void redo() {
        if (redoStack.empty()) {
            cout << "Nothing to redo." << endl;
            return;
        }

        // Save current state for undo
        undoStack.push(document);

        // Restore undone state
        document = redoStack.top();
        redoStack.pop();

        cout << "Redo performed." << endl;
    }

    // Display current document
    void display() {
        cout << "Document: " << document << endl;
    }
};
```

```
int main() {
    TextEditor editor;

    editor.makeChange("Hello");
    editor.display();

    editor.makeChange("Hello World");
    editor.display();

    editor.makeChange("Hello World!");
    editor.display();

    editor.undo();
    editor.display();

    editor.undo();
    editor.display();

    editor.redo();
    editor.display();

    return 0;
}
```

Output:

Change made.

Document: Hello

Change made.

Document: Hello World

Change made.

Document: Hello World!

Undo performed.

Document: Hello World

Undo performed.

Document: Hello

Redo performed.

Document: Hello World

Practical No. 04

Title: To implement a real-time event processing system using a Queue data structure and perform Add, Process, Display, and Cancel event operations.

```
#include <iostream>
#include <queue>
#include <string>
using namespace std;

int main()
{
   queue<string> q;
   int choice;
   string event;

   do
   {
      cout << "\n1.Add Event\n2.Process Event\n3.Display Events\n4.Cancel Event\n5.Exit\n";
      cout << "Enter choice: ";
      cin >> choice;

      switch(choice)
      {
         case 1:
            cout << "Enter event: ";
            cin >> event;
```

```
    q.push(event);
    cout << "Event added.\n";
    break;

case 2:
    if(q.empty())
        cout << "Queue is empty.\n";
    else
    {
        cout << "Processed: " << q.front() << endl;
        q.pop();
    }
    break;

case 3:
    if(q.empty())
        cout << "No pending events.\n";
    else
    {
        queue<string> temp = q;
        cout << "Pending Events: ";
        while(!temp.empty())
        {
            cout << temp.front() << " ";
            temp.pop();
```

```
            }
            cout << endl;
        }
        break;

    case 4:
        cout << "Enter event to cancel: ";
        cin >> event;
        {
            queue<string> temp;
            while(!q.empty())
            {
                if(q.front() != event)
                    temp.push(q.front());
                q.pop();
            }
            q = temp;
            cout << "Event cancelled.\n";
        }
        break;
    }
  } while(choice != 5);

  return 0;
}
```

Output:

1. Add Event

2. Process Event

3. Display Events

4. Cancel Event

5. Exit

Enter choice: 1

Enter event: Payment

Event added.

Enter choice: 1

Enter event: Login

Event added.

Enter choice: 1

Enter event: Order

Event added.

Enter choice: 3

Pending Events: Payment Login Order

Enter choice: 2

Processed: Payment

Enter choice: 4

Enter event to cancel: Login

Event cancelled.

Enter choice: 3

Pending Events: Order

Enter choice: 5

Practical No: 5(C)

Title: Implement a hash table of size 10 and use the division method as a hash function. In case of a collision, use chaining.

```
#include <iostream>
#include <list>
using namespace std;

int main()
{
    list<int> ht[10];
    int a[] = {23, 43, 13, 27, 37, 48};
    int n = 6;

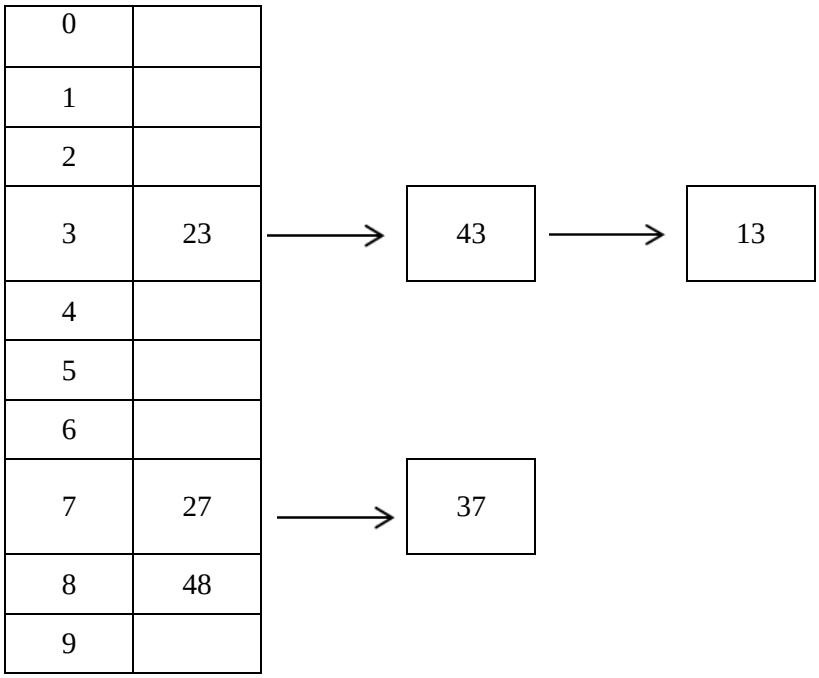
    for(int i = 0; i < n; i++)
        ht[a[i] % 10].push_back(a[i]);

    cout << "Hash Table:\n";
    for(int i = 0; i < 10; i++)
    {
        cout << i << " : ";
        for(int x : ht[i])
            cout << x << " -> ";
        cout << "NULL\n";
    }
```

```
    return 0;
}
```

Output:

Hash Table:



Hash function: key % 10

Collision handling: Chaining

Practical No. 6(C)

Title:

Design and implement a hash table of fixed size. Use the division method for the hash function and resolve collisions using linear probing. Allow the user to perform the following operations:

• Insert a key

• Search for a key

• Delete a key

• Display the table

Program:

```
#include <iostream>

using namespace std;


#define SIZE 10

int h[SIZE];


void insert(int k) {

   int i = k % SIZE;

   while (h[i] != -1)

      i = (i + 1) % SIZE;

   h[i] = k;

}


void search(int k) {

   int i = k % SIZE;
```

```
    while (h[i] != -1) {
        if (h[i] == k) {
            cout << k << " found at " << i << endl;
            return;
        }
        i = (i + 1) % SIZE;
    }
    cout << k << " not found\n";
}

void del(int k) {
    int i = k % SIZE;
    while (h[i] != -1) {
        if (h[i] == k) {
            h[i] = -1;
            cout << k << " deleted\n";
            return;
        }
        i = (i + 1) % SIZE;
    }
}

void display() {
    for (int i = 0; i < SIZE; i++)
        cout << i << " : " << h[i] << endl;
```

```
}

int main() {
    for (int i = 0; i < SIZE; i++) h[i] = -1;

    insert(25);
    insert(35);
    insert(15);
    insert(42);

    cout << "Hash Table:\n";
    display();

    search(35);
    del(25);

    cout << "\nAfter deletion:\n";
    display();

    return 0;
}
```

Output:

Hash Table:

0	-1
1	-1
2	42
3	-1
4	-1
5	25
6	35
7	15
8	-1
9	-1

35 found at 6

25 deleted

After deletion:

0	-1
1	-1
2	42
3	-1
4	-1
5	-5
6	35
7	15
8	-1
9	-1

Practical No. 7(D)

Title: Consider a particular area in your city. Note the popular locations A, B, C . . . in that area. Assume these locations represent nodes of a graph. If there is a route between two locations, it is represented as connections between nodes. Find out the sequence in which you will visit thes locations, starting from (say A) using (i) BFS and (ii) DFS. Represent a given graph using an adjacency matrix to perform DFS and an adjacency list to perform BFS

Program:

```
#include <stdio.h>
#define N 6
// DFS using Adjacency Matrix
void DFS(int graph[N][N], int visited[], int v)
{
}
int i;
printf("%c ", v + 'A');
visited[v] = 1;
for (i = 0; i < N; i++)
{
}
if (graph[v][i] == 1 && visited[i] == 0)
DFS(graph, visited, i);
// BFS using Adjacency List
void BFS(int adj[N][N], int start)
{
int queue[N], visited[N] = {0};
```

```
int front = 0, rear = 0;
int i, v;

    queue[rear++] = start;
    visited[start] = 1;

    while (front < rear)
    {
        v = queue[front++];
        printf("%c ", v + 'A');

        for (i = 0; i < N; i++)
        {
            if (adj[v][i] == 1 && visited[i] == 0)
            {
                queue[rear++] = i;
                visited[i] = 1;
            }
        }
    }
}

int main()
{
    int graph[N][N] = {
```

```
    {0,1,1,0,0,0},
    {1,0,0,1,1,0},
{1,0,0,0,0,1},
{0,1,0,0,1,0},
{0,1,0,1,0,1},
{0,0,1,0,1,0}
};
int visited[N] = {0};
printf("DFS Traversal: ");
DFS(graph, visited, 0);
printf("\nBFS Traversal: ");
BFS(graph, 0);
return 0;
}
```

Output:

DFS Traversal: A B D E F C

BFS Traversal: A B C D E F

Practical No. 8(D)

Title: Implement various operations on a Binary Search Tree, such as insertion, deletion, display, and search.

Program:

```
#include <stdio.h>

#include <stdlib.h>


struct Node {

   int data;

   struct Node *left, *right;

};


struct Node* newNode(int x) {

   struct Node *p = malloc(sizeof(struct Node));

   p->data = x;

   p->left = p->right = NULL;

   return p;

}


struct Node* insert(struct Node *root, int x) {

   if (root == NULL) return newNode(x);


   if (x < root->data)

      root->left = insert(root->left, x);

   else if (x > root->data)
```

```
        root->right = insert(root->right, x);

    return root;
}

void inorder(struct Node *root) {
    if (root) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

struct Node* search(struct Node *root, int x) {
    if (root == NULL || root->data == x)
        return root;

    if (x < root->data)
        return search(root->left, x);
    return search(root->right, x);
}

struct Node* deleteNode(struct Node *root, int x) {
    struct Node *temp;
```

```
if (root == NULL) return root;


if (x < root->data)
    root->left = deleteNode(root->left, x);
else if (x > root->data)
    root->right = deleteNode(root->right, x);
else {
    if (root->left == NULL) {
        temp = root->right;
        free(root);
        return temp;
    }
    if (root->right == NULL) {
        temp = root->left;
        free(root);
        return temp;
    }


    temp = root->right;
    while (temp->left)
        temp = temp->left;


    root->data = temp->data;
    root->right = deleteNode(root->right, temp->data);
}
```

```
    return root;
}

int main() {
    struct Node *root = NULL;

    root = insert(root, 50);
    insert(root, 30);
    insert(root, 70);
    insert(root, 20);
    insert(root, 40);
    insert(root, 60);
    insert(root, 80);

    printf("BST: ");
    inorder(root);

    printf("\nSearch 40: ");
    printf(search(root, 40) ? "Found" : "Not Found");

    root = deleteNode(root, 30);

    printf("\nAfter deletion: ");
    inorder(root);
```

```
    return 0;
}
```

Output:

BST: 20 30 40 50 60 70 80

Search 40: Found

After deletion: 20 40 50 60 70 80